Name: Abdulaziz Mohammed Alkhlawi
ID: 2240006095

Lab2: Text Pre-processing and Regular Expressions

Completed lab solution. Original instructions: NLP_Lab2.

Run the cells from top to bottom. Results below were generated by executing this notebook. The `dataset` folder contains the original data needed by this lab. Missing Python packages are installed by the setup cell; pretrained model downloads need internet on their first run. Random seeds are fixed where supported.

```
# Install only missing dependencies into the active Jupyter Python environment.
import importlib.util, subprocess, sys, os, random
from pathlib import Path
packages = {'numpy': 'numpy', 'pandas': 'pandas', 'matplotlib': 'matplotlib', 'sklearn': 'scikit-learn',
'nltk': 'nltk', 'spacy': 'spacy'}
missing = [package for module, package in packages.items()
           if importlib.util.find_spec(module) is None]
if missing:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', *missing])
os.environ['TOKENIZERS_PARALLELISM'] = 'false'
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
os.environ['USE_TF'] = '0'  # Transformers examples use PyTorch.
os.environ['USE_FLAX'] = '0'
import numpy as np
import pandas as pd
from IPython.display import display, Markdown, HTML
random.seed(42)
np.random.seed(42)
pd.set_option('display.max_colwidth', 90)
pd.set_option('display.max_columns', 10)
DATA = Path('dataset')
assert DATA.exists() or 2 in (7, 8, 9), 'Open this notebook from its Lab folder.'
```

In this lab, you will learn essential techniques for preparing and cleaning text data using various pre-processing steps. Additionally, you will gain hands-on experience with regular expressions, a powerful tool for pattern matching and text manipulation in natural language processing (NLP) tasks.

Regular Expressions

Regular expressions (regex) extremely useful in extracting information from any text by searching for one or more matches of a specific search pattern

https://www.regexpal.com/

Regular Expressions use cases

- Data pre-processing
- Pattern matching
- Text feature Engineering
- Web scraping
- Data validation
- Data extraction

An example use case is extracting all hashtags from a tweet, or getting email addresses or phone numbers from large unstructured text content.

Regular Expressions with Python

Python provides a convenient built-in module for managing regular expressions:

> import re

We can import it as we import any other Python library.

Important symbols in Regular Expression (For your reference)

Let's start with the basic regular expression characters and some examples:

- (^) : Matches the expression to its right, at the start of a string before it finds a line break.
- ($) : Matches the expression to its left, at the end of a string before it finds a line break.
- (.) : Matches any character except newline.
- (a) : Matches exactly one character.
- (ab) : Matches the string ab.

Quantifiers:

- (a|b) : Matches expression a or b. If a is matched first, b is not checked;
- (+) : Matches the expression to its left 1 or more times;
- (*) : Matches the expression to its left 0 or more times;
- (?) : Matches the expression to its left 0 or 1 times.

Character Classes:

- \w : Matches alphanumeric characters, that is a-z, A-Z, 0–9, and underscore(_);
- \W : Matches non-alphanumeric characters, that is except a-z, A-Z, 0–9, and _;
- \d : Matches digits, from 0–9;
- \D : Matches any non-digits;
- \s : Matches whitespace characters, which also include the \t, \n, \r, and space characters;
- \S : Matches non-whitespace characters.
- \n : Matches a newline character;
- \t : Matches a tab character;
- \b : Matches the word boundary (or empty string) at the start and end of a word;
- \B : Matches where \b does not, that is non-word boundary.

Sets:

- [abc] : Matches either a, b, or c. It does not match abc;
- [a-z] : Matches any alphabet from a to z;
- [A-Z] : Matches any alphabets in capital from A to Z;
- [a-p] : Matches a, -, or p. It matches - because \ escapes it;
- [-z] : Matches - or z;
- [a-z0–9] : Matches characters from a to z or from 0 to 9.
- [(+*)] : Special characters become literal inside a set, so this matches (, +, *, or);

- [^ab5] : Adding ^ excludes any character in the set. Here, it matches characters that are not a, b, or 5;

Regular Expression functions

search

Search for pattern occurrences in a string using the search function of the **re module**. This function returns a match object, containing the first matched substring (or None, if it doesn't exist) and its position inside the original string.

```
import re

text = 'I am enjoying the NLP course.'

print(re.search("I", text)) #r"I" means it's a raw string literal
print(re.search("se.$", text)) #$ Search at the end of the string
print(re.search("am", text))
print(re.search("m", text))
print(re.search("AI", text))
```

```
<re.Match object; span=(0, 1), match='I'>
<re.Match object; span=(26, 29), match='se.'>
<re.Match object; span=(2, 4), match='am'>
<re.Match object; span=(3, 4), match='m'>
None
```

match

The match function is similar to search, but it only tries to match the pattern at the beginning of the target string.

```
print(re.match.__doc__)
```

```
Try to apply the pattern at the start of the string, returning
    a Match object, or None if no match was found.
```

```
import re

text = 'I am enjoying the NLP course.'

print(re.search("I.*", text)) #global
print(re.match("I.*", text)) #local >> search at the start of the string

print(re.search("enjoying.*", text))
print(re.match("enjoying.*", text))
```

```
<re.Match object; span=(0, 29), match='I am enjoying the NLP course.'>
<re.Match object; span=(0, 29), match='I am enjoying the NLP course.'>
<re.Match object; span=(5, 29), match='enjoying the NLP course.'>
None
```

```
import re

text = '1999 was the year I was born.'

print(re.search(r"[a-zA-Z]+", text))
print(re.match(r"[a-zA-Z]+", text))
```

```
<re.Match object; span=(5, 8), match='was'>
None
```

Search does a global search, whereas match does a local search!

Match is often used when you need to check if the string starts with a specific pattern.

Search is used when you want to find a pattern anywhere within the string.

findall

The findall function looks for all the pattern matches in the target string (whereas search and match look only for the first occurrence).

- `re.findall()` returns a list of all matches.
- `re.finditer()` returns an iterator of match objects.
- `re.search() and re.match()` both return a single match object for the first occurrence (but differ in where they start matching: match() matches only at the start of the string, search() anywhere).

```
import re

text1 = "a 1 b 2 c 3"
text2 = "a1 b2 c 3"
text3 = "a 1 b 222 c 3"

print(re.findall(r"\d+", text1)) #match any digit
print(re.findall(r"\d+", text2))
print(re.findall(r"\d+", text3))
```

```
['1', '2', '3']
['1', '2', '3']
['1', '222', '3']
```

sub

The sub is a function that finds patterns in the target string and substitute them with another string.

flags=re.I --> non-case-sensitive

count = specify number of matches you want to replace

```
print(re.sub.__doc__)
```

```
Return the string obtained by replacing the leftmost
    non-overlapping occurrences of the pattern in string by the
    replacement repl.  repl can be either a string or a callable;
    if a string, backslash escapes in it are processed.  If it is
    a callable, it's passed the Match object and must return
    a replacement string to be used.
```

```
import re

text = 'I am enjoying the Math course.'

print(re.sub(r"Math", "NLP", text, count=1, flags=re.I)) #flags=re.I >> Ignore the case (no matter if M or m)
                                                         # count = 1 >> number of word I need it to change. count= 0
by default
```

```
I am enjoying the NLP course.
```

Text pre-processing steps

Text preprocessing involves transforming text into a clean and consistent format that can then be fed into a model for further analysis and learning. Raw text data might contain unwanted or unimportant text due to which our results might not give efficient accuracy, and might make it hard to understand and analyze.

The various text preprocessing steps are:

1. Tokenization.
2. Lower casing.
3. Stop words removal.
4. Stemming.
5. Lemmatization.

Before implementing the pre-processing, let us understand the concept first.

Tokenization

Tokenization is used in NLP to split paragraphs and sentences into smaller units that can be more easily assigned meaning.

The first step of the NLP process is gathering the data (a sentence) and breaking it into understandable parts (words).

- **Sentence Tokenization**

```
import nltk
for resource in ['punkt_tab', 'stopwords', 'wordnet', 'omw-1.4', 'averaged_perceptron_tagger_eng']:
    nltk.download(resource, quiet=True)
```

```
import nltk

text = "I'm enjoying the NLP course! I am also learning new concepts."

print(nltk.sent_tokenize(text))
```

```
["I'm enjoying the NLP course!", 'I am also learning new concepts.']
```

- **Word Tokenzitaion**

```
import nltk

text = "I'm enjoying the NLP course!"

print(nltk.word_tokenize(text)) #I'm is tokenized to two tokens [I,m], course! is also tokenized to two
tokens [course,!]
```

```
['I', "'m", 'enjoying', 'the', 'NLP', 'course', '!']
```

```
text = "I'm enjoying the NLP course!"
text.split()
```

```
["I'm", 'enjoying', 'the', 'NLP', 'course!']
```

Lower Casing

Converting a word to lower case (NLP -> nlp). Words like Book and book mean the same but when not converted to the lower case those two are represented as two different words in the vector space model (resulting in more dimensions).

```
text = "I'm enjoying the NLP course!"
text = text.lower()
print(text)
```

```
i'm enjoying the nlp course!
```

Stemming

Stemming is basically removing the suffix from a word and reduce it to its root word. For example: “Flying” is a word and its suffix is “ing”, if we remove “ing” from “Flying” then we will get base word or root word which is “Fly”. We uses these suffix to create a new word from original stem word.

https://www.nltk.org/howto/stem.html

What is PorterStemmer?

It is one of the most popular stemming methods proposed in 1980. It is based on the idea that the suffixes in the English language are made up of a combination of smaller and simpler suffixes. This stemmer is known for its speed and simplicity. The main applications of Porter Stemmer include data mining and Information retrieval. However, its applications are only limited to English words. Also, the group of stems is mapped on to the same stem and the output stem is not necessarily a meaningful word. The algorithms are fairly lengthy in nature and are known to be the oldest stemmer.

> Advantage: It produces the best output as compared to other stemmers and it has less error rate.

> Limitation: Morphological variants produced are not always real words.

```
import nltk
from nltk.stem import PorterStemmer

ps = PorterStemmer() # create a PorterStemmer object

words = ['run','runner','running','ran','runs','easily','fairly']

for word in words:
    print(word + ' --> ' + ps.stem(word))
```

```
run --> run
runner --> runner
running --> run
ran --> ran
runs --> run
easily --> easili
fairly --> fairli
```

What is SnowballStemmer?

It is a stemming algorithm which is also known as the Porter2 stemming algorithm as it is a better version of the Porter Stemmer since some issues of it were fixed in this stemmer.

> Advantage: It is slightly faster computation time than porter, with a reasonably large community around it.

```
import nltk
from nltk.stem.snowball import SnowballStemmer

sn = SnowballStemmer(language='english')

words = ['run','runner','running','ran','runs','easily','fairly']

for word in words:
    print(word+' --> '+sn.stem(word))
```

```
run --> run
runner --> runner
running --> run
ran --> ran
runs --> run
easily --> easili
fairly --> fair
```

Comparing between porter and snowball

```
words = ['generous','generation','generously','generate']

for word in words:
    print(word+' --> '+ps.stem(word))
    print(word+' --> '+sn.stem(word))
    print('----------------------------------------')
```

```
generous --> gener
generous --> generous
----------------------------------------
generation --> gener
generation --> generat
----------------------------------------
generously --> gener
generously --> generous
----------------------------------------
generate --> gener
generate --> generat
----------------------------------------
```

Note: Spacy does not provide stemming

Lemmatization

Lemmatization is the process of grouping together the different inflected forms of a word so they can be analyzed as a single item. Lemmatization is similar to stemming but it brings context to the words. So it links words with similar meanings to one word.

> One major difference with stemming is that lemmatize takes a part of speech parameter (pos) If not supplied, the default is “noun.”

Part of Speech	Description	Example Words
Noun (N)	Person, place, thing, idea	dog, city, love, Python
Verb (V)	Action or state	run, eat, is, was, jump
Adjective (ADJ)	Describes a noun	happy, blue, fast

Part of Speech	Description	Example Words
Adverb (ADV)	Describes a verb/adjective	quickly, very, well
Pronoun (PRON)	Replaces a noun	he, she, it, they
Preposition (ADP)	Shows relationship	in, on, under, with
Conjunction (CONJ)	Connects words/clauses	and, but, or
Determiner (DET)	Introduces a noun	a, an, the
Interjection (INTJ)	Expresses emotion	wow, oh, hey

```
#nltk.download('wordnet')
```

```
import nltk
from nltk.stem import WordNetLemmatizer

text = 'I am enjoying AI courses. I am taking NLP course.'
lemmatizer = WordNetLemmatizer()

for word in nltk.word_tokenize(text):
    print(f"{word}: ", lemmatizer.lemmatize(word))
```

```
I:  I
am:  am
enjoying:  enjoying
AI:  AI
courses:  course
.:  .
I:  I
am:  am
taking:  taking
NLP:  NLP
course:  course
.:  .
```

```
# Changing pos to verb
for word in nltk.word_tokenize(text):
    print(f"{word}: ", lemmatizer.lemmatize(word, pos='v'))
```

```
I:  I
am:  be
enjoying:  enjoy
AI:  AI
courses:  course
.:  .
I:  I
am:  be
taking:  take
NLP:  NLP
course:  course
.:  .
```

You may go through Spacy's lemmatizer: https://spacy.io/api/lemmatizer

Stop Words Removal

The words which are generally filtered out before processing a natural language are called stop words. These are actually the most common words in any language (like articles, prepositions, pronouns, conjunctions, etc) and does not add much information to the text. Examples of a few stop words in English are “the”, “a”, “an”, “so”, “what”.

Why do we need to remove stopwords?

By removing these words, we remove the low-level information from our text in order to give more focus to the important information. In order words, we can say that the removal of such words does not show any negative consequences on the model we train for our task.

- reduces the dataset size
- reduces the training time

Do we always remove stopwords? **NO!**

We do not always remove the stop words. The removal of stop words is highly dependent on the task we are performing and the goal we want to achieve. For example, if we are training a model that can perform the sentiment analysis task, we might not remove the stop words.

Movie review: “The movie was not good at all.”

Text after removal of stop words: “movie good”

```
import spacy
if not spacy.util.is_package('en_core_web_sm'):
    subprocess.check_call([sys.executable, '-m', 'spacy', 'download', 'en_core_web_sm'])
nlp = spacy.load('en_core_web_sm')
```

```
print(sorted(nlp.Defaults.stop_words))
print(len(nlp.Defaults.stop_words))
```

```
["'d", "'ll", "'m", "'re", "'s", "'ve", 'a', 'about', 'above', 'across', 'after', 'afterwards', 'again',
'against', 'all', 'almost', 'alone', 'along', 'already', 'also', 'although', 'always', 'am', 'among', 'amongst',
'amount', 'an', 'and', 'another', 'any', 'anyhow', 'anyone', 'anything', 'anyway', 'anywhere', 'are', 'around',
'as', 'at', 'back', 'be', 'became', 'because', 'become', 'becomes', 'becoming', 'been', 'before', 'beforehand',
'behind', 'being', 'below', 'beside', 'besides', 'between', 'beyond', 'both', 'bottom', 'but', 'by', 'ca',
'call', 'can', 'cannot', 'could', 'did', 'do', 'does', 'doing', 'done', 'down', 'due', 'during', 'each', 'eight',
'either', 'eleven', 'else', 'elsewhere', 'empty', 'enough', 'even', 'ever', 'every', 'everyone', 'everything',
'everywhere', 'except', 'few', 'fifteen', 'fifty', 'first', 'five', 'for', 'former', 'formerly', 'forty', 'four',
'from', 'front', 'full', 'further', 'get', 'give', 'go', 'had', 'has', 'have', 'he', 'hence', 'her', 'here',
'hereafter', 'hereby', 'herein', 'hereupon', 'hers', 'herself', 'him', 'himself', 'his', 'how', 'however',
'hundred', 'i', 'if', 'in', 'indeed', 'into', 'is', 'it', 'its', 'itself', 'just', 'keep', 'last', 'latter',
'latterly', 'least', 'less', 'made', 'make', 'many', 'may', 'me', 'meanwhile', 'might', 'mine', 'more',
'moreover', 'most', 'mostly', 'move', 'much', 'must', 'my', 'myself', "n't", 'name', 'namely', 'neither',
'never', 'nevertheless', 'next', 'nine', 'no', 'nobody', 'none', 'noone', 'nor', 'not', 'nothing', 'now',
'nowhere', 'n‘t', 'n’t', 'of', 'off', 'often', 'on', 'once', 'one', 'only', 'onto', 'or', 'other', 'others',
'otherwise', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 'part', 'per', 'perhaps', 'please', 'put',
'quite', 'rather', 're', 'really', 'regarding', 'same', 'say', 'see', 'seem', 'seemed', 'seeming', 'seems',
'serious', 'several', 'she', 'should', 'show', 'side', 'since', 'six', 'sixty', 'so', 'some', 'somehow',
'someone', 'something', 'sometime', 'sometimes', 'somewhere', 'still', 'such', 'take', 'ten', 'than', 'that',
'the', 'their', 'them', 'themselves', 'then', 'thence', 'there', 'thereafter', 'thereby', 'therefore', 'therein',
'thereupon', 'these', 'they', 'third', 'this', 'those', 'though', 'three', 'through', 'throughout', 'thru',
'thus', 'to', 'together', 'too', 'top', 'toward', 'towards', 'twelve', 'twenty', 'two', 'under', 'unless',
'until', 'up', 'upon', 'us', 'used', 'using', 'various', 'very', 'via', 'was', 'we', 'well', 'were', 'what',
'whatever', 'when', 'whence', 'whenever', 'where', 'whereafter', 'whereas', 'whereby', 'wherein', 'whereupon',
'wherever', 'whether', 'which', 'while', 'whither', 'who', 'whoever', 'whole', 'whom', 'whose', 'why', 'will',
'with', 'within', 'without', 'would', 'yet', 'you', 'your', 'yours', 'yourself', 'yourselves', '‘d', '‘ll', '‘m',
'‘re', '‘s', '‘ve', '’d', '’ll', '’m', '’re', '’s', '’ve']
326
```

```
print(nlp.vocab['myself'].is_stop)
print(nlp.vocab['mystery'].is_stop)
```

```
True
False
```

```
print (nlp.vocab['btw'].is_stop) #print False
```

```
False
```

```
nlp.Defaults.stop_words.add('btw')

nlp.vocab['btw'].is_stop = True # You must update the is_stop flag for the word you added

print (nlp.vocab['btw'].is_stop)
```

```
True
```

```
nlp.Defaults.stop_words.remove('beyond')
nlp.vocab['beyond'].is_stop = False

print(nlp.vocab['beyond'].is_stop)
```

```
False
```

Stop words in NLTK

```
import nltk
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Azoz\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

```
True
```

```
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))
print(len(stop_words))
```

```
198
```

```
print(sorted(stop_words))
```

```
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren',
"aren't", 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can',
'couldn', "couldn't", 'd', 'did', 'didn', "didn't", 'do', 'does', 'doesn', "doesn't", 'doing', 'don', "don't",
'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'hadn', "hadn't", 'has', 'hasn', "hasn't",
'have', 'haven', "haven't", 'having', 'he', "he'd", "he'll", "he's", 'her', 'here', 'hers', 'herself', 'him',
'himself', 'his', 'how', 'i', "i'd", "i'll", "i'm", "i've", 'if', 'in', 'into', 'is', 'isn', "isn't", 'it',
"it'd", "it'll", "it's", 'its', 'itself', 'just', 'll', 'm', 'ma', 'me', 'mightn', "mightn't", 'more', 'most',
'mustn', "mustn't", 'my', 'myself', 'needn', "needn't", 'no', 'nor', 'not', 'now', 'o', 'of', 'off', 'on',
'once', 'only', 'or', 'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'shan',
"shan't", 'she', "she'd", "she'll", "she's", 'should', "should've", 'shouldn', "shouldn't", 'so', 'some', 'such',
't', 'than', 'that', "that'll", 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they',
"they'd", "they'll", "they're", "they've", 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 've',
'very', 'was', 'wasn', "wasn't", 'we', "we'd", "we'll", "we're", "we've", 'were', 'weren', "weren't", 'what',
'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', "won't", 'wouldn', "wouldn't",
'y', 'you', "you'd", "you'll", "you're", "you've", 'your', 'yours', 'yourself', 'yourselves']
```

Text pre-processing and Regular Expression Tasks:

Task 1:

To make you understand better about the need of text pre-processing. We will do a small project. In this project, you need to find how many hashtags and the top 10 hashtags used in the dataset.

The dataset is about people tweets about apple company on Twitter. You can find the dataset here:

https://www.kaggle.com/datasets/seriousran/appletwittersentimenttexts

```
from collections import Counter
tweets = pd.read_csv(DATA / 'apple-twitter-sentiment-texts.csv')
# Count hashtag occurrences in raw text before removing punctuation.
hashtags = [tag.casefold() for text in tweets['text'].dropna().astype(str)
            for tag in re.findall(r'(?<!\w)#[\w]+', text)]
counts = Counter(hashtags)
print('Tweets:', len(tweets))
print('Total hashtag occurrences:', len(hashtags))
print('Unique hashtags (case insensitive):', len(counts))
display(pd.DataFrame(counts.most_common(10), columns=['Hashtag','Occurrences']))
```

```
Tweets: 1630
Total hashtag occurrences: 1615
Unique hashtags (case insensitive): 555
```

	Hashtag	Occurrences
0	#aapl	400
1	#apple	162
2	#iphone	38
3	#december	37
4	#iphone6	29
5	#stocks	15
6	#iphone6plus	13
7	#ipad	13
8	#iphone6s	13
9	#tech	12

Task 2: Using re.compile()

Given the following text: "This year is 2021"

Write a Python program that:

1. Use re.compile() to create a regular expression pattern that matches one or more digits.
2. Print the type of the compiled pattern.
3. Use the compiled pattern to replace 2021 with 2022.
4. Print the updated text.
5. Briefly explain what re.compile() does and why it is useful when working with regular expressions.

```
text = 'This year is 2021'
pattern = re.compile(r'\d+')
print('Compiled pattern type:', type(pattern))
print('Updated text:', pattern.sub('2022', text))
```

```
Compiled pattern type: <class 're.Pattern'>
Updated text: This year is 2022
```

Task 3: Using re.split()

Given the following text = "a 11 b 2 3 c 4"

Write a Python program that:

1. Use re.split() with a regular expression pattern to split the text whenever one or more digits appear.
2. Print the resulting list.
3. Briefly explain: what re.split() does.

```
text = 'a 11 b 2 3 c 4'
print(re.split(r'\d+', text))
```

```
['a ', ' b ', ' ', ' c ', '']
```

Task 4: Tokenization Using spaCy

Processes the following sentence: "I'm enjoying the NLP course!"

Write a Python program that:

1. Use spaCy library and load the small English model en_core_web_sm.
2. Iterate through the text and print each token on a separate line.
3. Briefly explain: What spacy.load() does.
4. Tokenize the same text using nltk library and notice the differences compared to spaCy output.

```
text = "I'm enjoying the NLP course!"
print('spaCy tokens:')
for token in nlp(text):
    print(token.text)
spacy_tokens = [t.text for t in nlp(text)]
nltk_tokens = nltk.word_tokenize(text)
print('NLTK tokens:', nltk_tokens)
print('Identical tokenization:', spacy_tokens == nltk_tokens)
```

```
spaCy tokens:

I
'm
enjoying
the
NLP
course
!
NLTK tokens: ['I', "'m", 'enjoying', 'the', 'NLP', 'course', '!']
Identical tokenization: True
```

Explanations

Task 1: Hashtags must be extracted before punctuation removal, which would destroy the `#` marker. Case folding merges tags such as `#Apple` and `#apple`; repeated occurrences count separately.

Task 2: `re.compile` creates a reusable pattern object. Its methods apply the same regex repeatedly without rewriting the pattern and keep the matching configuration together.

Task 3: `re.split` cuts the string at each digit sequence. The original spaces remain, and the final separator produces an empty last item.

Task 4: `spacy.load` loads the model's tokenizer and processing pipeline. Both libraries split `I'm` into `I` and `'m` and separate `!` for this sentence. Other inputs can differ because their tokenization rules differ. Stopword removal is task dependent: removing negation can reverse sentiment.